

PW EARNERS
DSA in C++

DPP — 7 (SOLUTIONS)
Topic: 2D Arrays and 2D Vectors

Multiple Choice Questions (Predict the Output)

Q1. Array Initialization Output

Correct Option: **B (5 0)**

Explanation: The 2D array is initialized sequentially. Row 0 gets {1, 2, 3} and Row 1 gets {4, 5, 0} because missing elements default to 0. Thus, `arr[1][1]` is 5 and `arr[1][2]` is 0.

.....

Q2. Vector of Vectors Output

Correct Option: **D (7)**

Explanation: The vector is initialized with 3 rows and 2 columns, and every single element is initialized with the value 7. Printing any valid index will output 7.

.....

Q3. Passing 2D Array to Function

Correct Option: **B (Compilation Error)**

Explanation: In C++, when passing a 2D array as a parameter, you must specify the size of all dimensions except the first one (e.g., `int arr[][2]`). Leaving both brackets empty causes a compilation error.

.....

Q4. Modifying Vector Passed by Value

Correct Option: **B (1)**

Explanation: The 2D vector is passed by value to the `modify` function. A completely separate copy is created, so modifying the copy leaves the original vector in `main` unchanged.

.....

Q5. Pointer Arithmetic with 2D Array

Correct Option: **C (6)**

Explanation: `arr + 1` points to the second row. Dereferencing it `*(arr + 1)` gives the base address of the second row. Adding 2 moves the pointer to index 2 of that row (value 6). Dereferencing the whole expression prints 6.

.....

Q6. Crazy Pointer Output

Correct Option: **A (30 20)**

Explanation: `ptr` is a pointer to an array of 2 integers. `** (ptr + 1)` accesses the first element of the second row (30). `*( *ptr + 1)` accesses the second element of the first row (20).

.....

Q7. Dangling Reference on Push Back

Correct Option: **D (Undefined Behavior)**

Explanation: Pushing new elements into a vector often exceeds its capacity, forcing it to allocate new memory and move elements. The reference `ref` still points to the old, freed memory, becoming a dangling reference and causing undefined behavior.

.....

Q8. Row-wise Strictly Increasing

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  bool isStrictlyIncreasing(vector<vector<int>>& matrix, int m, int n)
6  {
7      for (int i = 0; i < m; i++) {
8          // Start from index 1 and compare with the previous element
9          for (int j = 1; j < n; j++) {
10             if (matrix[i][j] <= matrix[i][j - 1]) {
11                 return false;
12             }
13         }
14     }
15     return true;
16 }
17
18 int main() {
19     int m, n;
20     cin >> m >> n;
21     vector<vector<int>> matrix(m, vector<int>(n));
22     for (int i = 0; i < m; i++) {
23         for (int j = 0; j < n; j++) {
24             cin >> matrix[i][j];
25         }
26     }
27
28     if (isStrictlyIncreasing(matrix, m, n)) {
29         cout << "true\n";
30     } else {
31         cout << "false\n";
32     }
33 }

```

Time Complexity: $O(M \times N)$ | **Space Complexity:** $O(1)$

Explanation: We use nested loops to iterate through the matrix. For every row, we check if the current element is less than or equal to the previous element. If so, the row is not strictly increasing, and we immediately return false.

.....

Q9. Count Negatives in a Sorted Matrix

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;

```

```

4
5 int countNegatives(vector<vector<int>>& matrix, int m, int n) {
6     int count = 0;
7     // Start from the bottom-left corner
8     int row = m - 1;
9     int col = 0;
10
11     while (row >= 0 && col < n) {
12         if (matrix[row][col] < 0) {
13             // If the element is negative, all elements to its right
14             // are also negative
15             count += (n - col);
16             row--; // Move up to check the row above
17         } else {
18             col++; // Move right to find a negative number
19         }
20     }
21     return count;
22 }
23
24 int main() {
25     int m, n;
26     cin >> m >> n;
27     vector<vector<int>> matrix(m, vector<int>(n));
28     for (int i = 0; i < m; i++) {
29         for (int j = 0; j < n; j++) {
30             cin >> matrix[i][j];
31         }
32     }
33     cout << countNegatives(matrix, m, n) << endl;
34 }

```

Time Complexity: $O(M + N)$ | **Space Complexity:** $O(1)$

Explanation: Since the matrix is sorted in non-increasing order, we can begin our search at the bottom-left corner. If we find a negative number, every element to its right is guaranteed to be negative, so we add them to our count and move up. Otherwise, we move right.

Q10. Check Toeplitz Matrix

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 bool isToeplitzMatrix(vector<vector<int>>& matrix, int m, int n) {
6     // Check every element starting from the second row and second
7     // column
8     for (int i = 1; i < m; i++) {
9         for (int j = 1; j < n; j++) {
10             // It must be equal to its top-left neighbor
11             if (matrix[i][j] != matrix[i - 1][j - 1]) {
12                 return false;
13             }
14         }
15     }
16     return true;
17 }

```

```

14     }
15     return true;
16 }
17
18 int main() {
19     int m, n;
20     cin >> m >> n;
21     vector<vector<int>> matrix(m, vector<int>(n));
22     for (int i = 0; i < m; i++) {
23         for (int j = 0; j < n; j++) {
24             cin >> matrix[i][j];
25         }
26     }
27
28     if (isToeplitzMatrix(matrix, m, n)) {
29         cout << "true\n";
30     } else {
31         cout << "false\n";
32     }
33 }

```

Time Complexity: $O(M \times N)$ | **Space Complexity:** $O(1)$

Explanation: A matrix is Toeplitz if every diagonal has the same elements. We can verify this easily by checking if every element (except those in the first row or first column) matches the element exactly one step up and one step left (`matrix[i-1][j-1]`).

Q11. Rotate Matrix by 180 Degrees In-Place

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void rotate180(vector<vector<int>>& matrix, int n) {
6      // Step 1: Reverse the elements of each row
7      for (int i = 0; i < n; i++) {
8          int left = 0, right = n - 1;
9          while (left < right) {
10             int temp = matrix[i][left];
11             matrix[i][left] = matrix[i][right];
12             matrix[i][right] = temp;
13             left++; right--;
14         }
15     }
16
17     // Step 2: Swap the rows from top to bottom
18     int top = 0, bottom = n - 1;
19     while (top < bottom) {
20         for (int j = 0; j < n; j++) {
21             int temp = matrix[top][j];
22             matrix[top][j] = matrix[bottom][j];
23             matrix[bottom][j] = temp;
24         }
25         top++; bottom--;

```

```

26     }
27 }
28
29 int main() {
30     int n;
31     cin >> n;
32     vector<vector<int>> matrix(n, vector<int>(n));
33     for (int i = 0; i < n; i++) {
34         for (int j = 0; j < n; j++) {
35             cin >> matrix[i][j];
36         }
37     }
38
39     rotate180(matrix, n);
40
41     for (int i = 0; i < n; i++) {
42         for (int j = 0; j < n; j++) {
43             cout << matrix[i][j] << " ";
44         }
45         cout << endl;
46     }
47 }

```

Time Complexity: $O(N^2)$ | **Space Complexity:** $O(1)$

Explanation: Rotating a matrix by 180 degrees is equivalent to reversing every single row horizontally, and then swapping the top rows with the bottom rows vertically. We do this entirely in-place without needing extra space.

Q12. Find the Saddle Point

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void findSaddlePoint(vector<vector<int>>& matrix, int m, int n) {
6      for (int i = 0; i < m; i++) {
7          // Find the minimum element in the current row
8          int rowMin = matrix[i][0];
9          int colIndex = 0;
10
11         for (int j = 1; j < n; j++) {
12             if (matrix[i][j] < rowMin) {
13                 rowMin = matrix[i][j];
14                 colIndex = j;
15             }
16         }
17
18         // Verify if it is the maximum element in its column
19         bool isSaddle = true;
20         for (int k = 0; k < m; k++) {
21             if (matrix[k][colIndex] > rowMin) {
22                 isSaddle = false;
23                 break;

```

```

24         }
25     }
26
27     if (isSaddle) {
28         cout << rowMin << endl;
29         return;
30     }
31 }
32 cout << "-1" << endl;
33 }
34
35 int main() {
36     int m, n;
37     cin >> m >> n;
38     vector<vector<int>> matrix(m, vector<int>(n));
39     for (int i = 0; i < m; i++) {
40         for (int j = 0; j < n; j++) {
41             cin >> matrix[i][j];
42         }
43     }
44     findSaddlePoint(matrix, m, n);
45 }

```

Time Complexity: $O(M \times N)$ | **Space Complexity:** $O(1)$

Explanation: For every row, we first identify the minimum value and remember its column index. We then traverse that specific column to ensure no other number is greater than our minimum value. If it satisfies both conditions, it is our saddle point.

Q13. Shift 2D Grid by K Times

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void shiftGrid(vector<vector<int>>& grid, int m, int n, int k) {
6      int totalElements = m * n;
7      k = k % totalElements;
8
9      if (k == 0) return;
10
11     vector<vector<int>> temp(m, vector<int>(n));
12
13     for (int i = 0; i < m; i++) {
14         for (int j = 0; j < n; j++) {
15             // Mentally flatten the 2D matrix into a 1D array to
16             // calculate shift
17             int flatIndex = i * n + j;
18             int shiftedIndex = (flatIndex + k) % totalElements;
19
20             int newRow = shiftedIndex / n;
21             int newCol = shiftedIndex % n;
22
23             temp[newRow][newCol] = grid[i][j];

```

```

23     }
24 }
25
26 // Copy the shifted elements back into the original grid
27 for (int i = 0; i < m; i++) {
28     for (int j = 0; j < n; j++) {
29         grid[i][j] = temp[i][j];
30     }
31 }
32 }
33
34 int main() {
35     int m, n, k;
36     cin >> m >> n >> k;
37     vector<vector<int>> grid(m, vector<int>(n));
38     for (int i = 0; i < m; i++) {
39         for (int j = 0; j < n; j++) {
40             cin >> grid[i][j];
41         }
42     }
43
44     shiftGrid(grid, m, n, k);
45
46     for (int i = 0; i < m; i++) {
47         for (int j = 0; j < n; j++) {
48             cout << grid[i][j] << " ";
49         }
50         cout << endl;
51     }
52 }

```

Time Complexity: $O(M \times N)$ | **Space Complexity:** $O(M \times N)$

Explanation: The shifting behavior mimics a 1D array. We mathematically map the 2D coordinates to a 1D index, add K , and wrap it around using modulo. We then map the new 1D index back to 2D coordinates and place the value into a temporary array.

Q14. Anti-Diagonal Traversal

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void printAntiDiagonals(vector<vector<int>>& matrix, int m, int n) {
6      // The sum of row and col indices for anti-diagonals ranges from
7      // 0 to (m + n - 2)
8      int totalDiagonals = m + n - 1;
9
10     for (int sum = 0; sum < totalDiagonals; sum++) {
11         // Establish the starting coordinates for the current
12         // diagonal
13         int startRow = (sum < n) ? 0 : sum - n + 1;
14         int startCol = (sum < n) ? sum : n - 1;

```

```
14         // Traverse downwards and leftwards
15         while (startRow < m && startCol >= 0) {
16             cout << matrix[startRow][startCol] << " ";
17             startRow++;
18             startCol--;
19         }
20     }
21     cout << endl;
22 }
23
24 int main() {
25     int m, n;
26     cin >> m >> n;
27     vector<vector<int>> matrix(m, vector<int>(n));
28     for (int i = 0; i < m; i++) {
29         for (int j = 0; j < n; j++) {
30             cin >> matrix[i][j];
31         }
32     }
33     printAntiDiagonals(matrix, m, n);
34 }
```

Time Complexity: $O(M \times N)$ | **Space Complexity:** $O(1)$

Explanation: Every element in the same anti-diagonal shares the exact same sum of its row and column indices. We loop through all possible sums (from 0 to $M + N - 2$), identify the starting coordinates, and slide diagonally down-left (increasing the row, decreasing the column) until we go out of bounds.

All the best! Keep Coding ♡